

ÔN VẤN ĐÁP BẢO VỆ ĐỒ ÁN

Toàn bộ Repository

Mọi thứ hội đồng có thể hỏi về mã nguồn StoryLens

Monorepo 3 dịch vụ · Frontend · Backend · AI Module · CI/CD · Test · Deploy
Đọc kèm QA_DuyMinh.pdf (vai trò cá nhân) + QA_DuyMinh_ThuatNgu.pdf (thuật ngữ)

Môn học CSC10011 – Công nghệ phần mềm cho hệ thống Trí tuệ Nhân tạo
Nhóm Nhóm 1 – StoryLens · HCMUS · 10/07/2026

Contents

1	Tổng quan repository (monorepo)	2
2	Frontend (frontend/)	2
3	Backend (backend/) — tóm tắt	3
4	AI Module (ai_module/) — lõi ML	3
5	Huấn luyện & Mô hình (ai_module/training/)	4
6	Database & Migrations (backend/*.sql)	4
7	CI/CD & Quy trình Git	5
8	Kiểm thử (Testing)	5
9	Triển khai & Vận hành	6
10	Bảo mật toàn repo (tổng hợp)	6
11	Câu hỏi toàn repo & điểm yếu phải thử	6
12	Phụ lục — Cheat-sheet số liệu repo	7

1. Tổng quan repository (monorepo)

Repo được tổ chức thế nào? Vì sao gộp 3 dịch vụ trong một repo? [Cốt lõi]

Trả lời. Repo là **monorepo** gồm ba dịch vụ triển khai độc lập + tài liệu:

- frontend/ — Next.js 16 + React 19 + TypeScript (deploy Vercel).
- backend/ — FastAPI + Python 3.11, vai trò API Gateway (deploy Render, Docker).
- ai_module/ — FastAPI + PyTorch, chứa model nặng ~4GB (deploy HuggingFace Spaces).
- docs/ (SAD/SRS/SDP/PA1–PA5), supabase/ (config), render.yaml (IaC), .github/workflows/ci.yml.

Monorepo giúp một Pull Request thay đổi xuyên frontend + backend cùng lúc, review chung, phiên bản đồng bộ; nhưng mỗi thư mục vẫn **build & deploy độc lập**.

Ba dịch vụ giao tiếp với nhau ra sao? [Cốt lõi]

Trả lời. Frontend gọi Backend qua REST /v1/* (+ WebSocket /v1/ws/*); Backend gọi AI Module qua HTTP và gọi Gemini API; cả ba dùng chung Supabase (Postgres + Auth + Storage). Ảnh không truyền byte thô giữa backend và AI Module — AI Module tự tải ảnh từ URL Supabase Storage.

2. Frontend (frontend/)

Ngăn xếp frontend là gì? [Cốt lõi]

Trả lời. Next.js 16.2.6 (App Router) + React 19.2.4 + TypeScript + Tailwind + Framer Motion (animation). Kèm @sentry/nextjs (giám sát lỗi), pdfjs-dist (đọc PDF), react-easy-crop (cắt avatar), và hỗ trợ PWA đọc offline. Không dùng Redux/Zustand — chỉ React Context.

App Router là gì? Trang được tổ chức thế nào? [Cốt lõi]

Trả lời. App Router (Next.js) định tuyến theo **thư mục**: mỗi page.tsx trong src/app/<route>/ là một route. Repo có ~24 route: / (home), /login, /register, /upload, /reader, /qa, /library, /browse, /series, /forum, /history, /bookmarks, /plans, /settings, /profile/security, và nhóm /admin/* (analytics, users, content, health, model-monitor, audit, settings, ai-source).

Vì sao mọi lời gọi API đều qua src/lib/api.ts? [Cốt lõi]

Trả lời. api.ts là **fetch wrapper tập trung**: gom BASE_URL, gắn cookie credentials, **retry khi backend cold-start**, gắn **Idempotency-Key** cho thao tác ghi, và trả về **kiểu TypeScript** rõ ràng (UploadResponse, PageStatus, BubbleData...). Quy ước: **cấm gọi fetch() trực tiếp trong component** — để mọi request có cùng hành vi retry/lỗi/kiểu.

State toàn cục quản lý thế nào? [Cốt lõi]

Trả lời. Ba React Context: **AuthContext** (phiên đăng nhập, hydrate qua /auth/me), **WibuContext** (gamification: bookmark, rating, goal), **I18nContext** (song ngữ VI/EN lưu localStorage). Ba theme light/dark/sepia qua [data-theme] trên <html>.

Token đăng nhập lưu ở đâu trên frontend?

Trả lời. **Không lưu token trong localStorage**. Frontend không bao giờ thấy token — backend đặt access/refresh token vào **HTTP-only cookie**, JS không đọc được (chống XSS). Frontend chỉ biết “đã đăng

nhập” qua `/auth/me`. (`NEXT_PUBLIC_SUPABASE_ANON_KEY` chỉ dùng cho trang `/reset-password`.)

3. Backend (backend/) --- tóm tắt

Vai trò backend trong kiến trúc? [Cốt lõi]

Trả lời. API Gateway điều phối, **KHÔNG** chứa model ML. Chạy Render free tier 512MB nên không thể nạp model vài GB — toàn bộ việc nặng ủy thác cho AI Module qua HTTP. Backend: nhận request, gọi AI Module + Gemini, lưu Supabase, đẩy tiến trình qua WebSocket. Gồm **24 router** + **26 service**; router chỉ validate, business logic nằm ở `services/` (Layered Architecture).

Luồng upload-dịch một trang đi qua backend thế nào? [Cốt lõi]

Trả lời. `POST /v1/upload` → (1) kiểm Idempotency-Key; (2) kiểm credit + batch size; (3) validate từng ảnh (Pillow verify() chống giả MIME); (4) lưu Storage + tạo record `manga_pages`; (5) trừ 1 credit; (6) chạy **thread nền** giới hạn bởi **semaphore (tối đa 10 luồng)**. Thread gọi AI Module → lưu `bubble_data` + `translation_history` + `embeddings` (vector 768 chiều) → cập nhật status + đẩy WebSocket.

RAG Q&A chống rò rỉ dữ liệu chéo ở đâu trong code? [Cốt lõi]

Trả lời. Trong `services/rag.py`, hàm `answer_question` gọi RPC `match_embeddings` với tham số `filter_user_id` — pgvector chỉ tìm trong truyện của đúng người dùng đó (đóng lỗ hổng IDOR), kèm ngưỡng cosine `RAG_MIN_SIMILARITY = 0.55`. Câu trả lời kèm nguồn trích; Gemini chết thì trả extractive, không bịa.

4. AI Module (ai_module/) --- lõi ML

AI Module là gì? Chứa những gì? [Cốt lõi]

Trả lời. Dịch vụ FastAPI **tự chứa** (self-contained): `main.py` nạp `server.main`, in ra một **nonce** (khóa phiên) và expose `/health`. Bên trong `manga_translator/` là pipeline 5 bước, mặc định: `detector manga_yolo` → OCR `mocr (manga-ocr)` → inpainter `lama_large` → translator `gemini` → renderer `manga2eng_pillow`. Ảnh Docker ~4GB, chạy **CPU-only** trên HF Spaces free tier.

Pipeline dịch 5--6 bước cụ thể? [Cốt lõi]

Trả lời. (1) **YOLOv8** phát hiện bong bóng → `bbox`; (2) **manga-ocr** đọc chữ Nhật trong từng `bbox`; (3) **LaMa** inpaint xóa chữ gốc khỏi ảnh; (4) **Gemini** dịch có ngữ cảnh sang tiếng Việt; (5) **renderer** vẽ chữ Việt vào đúng khung (binary-search chọn cỡ chữ vừa khít); (6) backend sinh **embedding** lưu pgvector cho RAG.

Vì sao AI Module tách riêng, không nhét vào backend? [Cốt lõi]

Trả lời. Model (YOLOv8, manga-ocr, LaMa) nặng vài GB, cần RAM lớn; backend Render free tier chỉ 512MB không tải nổi. Tách riêng → cô lập lỗi (AI Module sập, người dùng vẫn đọc truyện đã dịch + forum), và scale độc lập (đem AI Module lên GPU khi cần). Đánh đổi: thêm latency HTTP — khắc phục bằng WebSocket báo tiến trình thời gian thực.

5. Huấn luyện & Mô hình (ai_module/training/)

Code huấn luyện nằm ở đâu? Có phải nhóm tự train thật không? [Cốt lõi]

Trả lời. Có, code thật trong ai_module/training/manga109/: 01_parse_visualize.py (đọc annotation Manga109-s), 02_convert_to_yolo.py (chuyển bbox <text> sang nhãn YOLO 1 lớp), 03_train_ocr.py (fine-tune OCR end-to-end). Chạy trên **Kaggle GPU T4**.

Fine-tune OCR cụ thể trong code làm gì? [Cốt lõi]

Trả lời. 03_train_ocr.py: [1] trích cặp (crop + text) từ XML Manga109-s, **split theo title 80/10/10** chống leak, lọc bbox nhỏ/text rỗng/quá dài; [2] fine-tune **VisionEncoderDecoderModel** (kha-white/manga-ocr-base) bằng HuggingFace **Seq2SeqTrainer**, **AdamW + cosine LR + label smoothing + fp16**; [3] đánh giá **CER + char-acc + exact-match** trên test; [5] lưu model + metadata.json; [6] tùy chọn push lên HuggingFace Hub cho production.

Kết quả huấn luyện? [Cốt lõi]

Trả lời. OCR: **CER 6,1%** (char-acc 93,9%, exact-match 67,5%); ablation **PA2 12,7% → PA3 6,1%**, xác minh bằng metadata.json (test_cer = 0.061090). YOLOv8: **mAP@0.5 = 95,3%**, mAP@0.5:0.95 = 84,4%, Precision 97,3%, Recall 93,8%, ngưỡng 85% — đều vượt. Test split: 956 ảnh / 14.130 vùng chữ.

Điểm yếu về artifact mô hình? [Trung thực]

Trả lời. YOLOv8 **không còn artifact gốc** (results.csv/PR-curve), chỉ có best.pt; số mAP dựa vào SAD PA3 — giới hạn minh bạch đã ghi nhận. OCR thì có metadata.json đối chiếu được. Hướng khắc phục: lưu results.csv khi train lại.

6. Database & Migrations (backend/*.sql)

Cơ sở dữ liệu dùng gì? Có extension gì đặc biệt? [Cốt lõi]

Trả lời. **Supabase PostgreSQL** + extension **pgvector** (lưu/tìm vector RAG 768 chiều) + **pg_trgm** (tìm kiếm mờ theo chuỗi). Kèm Supabase Auth + Storage (4 bucket: manga-originals, manga-thumbnails private; avatars public read; manga-audio narration).

Quản lý thay đổi schema thế nào? [Cốt lõi]

Trả lời. ~18 file migration **đánh số phiên bản** chạy tuần tự: base → credits → series → wibu → admin → v2...v9. Mọi migration **idempotent** (IF NOT EXISTS) nên chạy lại an toàn. Đáng chú ý: v7_rag (cột vector(768) + RPC match_embeddings owner-scoped), v8_model_monitor, v9_ai_call_metrics. Render free tier không có pre-deploy hook nên **migration chạy tay** trước khi push (python -m app.scripts.apply_migrations).

Bảo mật mức dữ liệu (RLS) hoạt động thế nào? [Cốt lõi]

Trả lời. **Row Level Security** trên Supabase: mỗi bảng dữ liệu người dùng có owner_id/user_id; policy tự thêm điều kiện = auth.uid() vào mọi truy vấn → user A không đọc được dữ liệu user B ngay ở tầng DB, kể cả nếu tầng ứng dụng có sơ hở.

7. CI/CD & Quy trình Git

CI chạy những gì? [Cốt lõi]

Trả lời. .github/workflows/ci.yml chạy **5 job song song** trên GitHub Actions: (1) **test** = `tsc --noEmit + vitest`; (2) **lint** = ESLint; (3) **build** = `next build`; (4) **e2e** = Playwright (chromium + mobile); (5) **backend** = `pytest`. Chỉ khi tất cả xanh + có review thì mới merge main.

(BẤY) CI có chạy khi CHỈ sửa backend không?

Trả lời. Trung thực: workflow cấu hình paths: `frontend/**` nên **CI chỉ trigger khi có thay đổi trong frontend/** (hoặc chính file workflow). Thay đổi **chỉ ở backend** hiện KHÔNG tự kích hoạt CI. Job backend `pytest` nằm trong cùng workflow nên chạy kèm khi frontend đổi. **Nếu bị hỏi:** thừa nhận đây là điểm cần cải thiện — nên thêm `backend/**` vào paths để bảo vệ backend độc lập. (Đừng khẳng định “mọi push backend đều chạy CI”).

Nhóm dùng chiến lược nhánh (branch) nào? [Cốt lõi]

Trả lời. Nhánh chính main (production, auto-deploy). CI trigger trên main, dev, và nhánh tính năng cá nhân (vd duong). Mọi thay đổi vào main phải qua **Pull Request + review + CI xanh** — không push thẳng. Đây là thực hành XP (Continuous Integration + code review).

8. Kiểm thử (Testing)

Nhóm kiểm thử ở những tầng nào? (test pyramid) [Cốt lõi]

Trả lời. Theo hình chóp kiểm thử — nhiều unit, ít E2E:

- **Frontend unit/component:** Vitest + React Testing Library (`frontend/__tests__`): unit, components, integration).
- **Frontend E2E:** Playwright (`frontend/e2e/`): `auth`, `home`, `protected-pages`, `edge-cases`, `mobile`, `forum`, `narration`.
- **Backend:** `pytest` 14 file (`test_rag`, `test_recap`, `test_idempotency`, `test_scraper`, `test_forum`, `narration ×3`, `test_tts_registry`, `test_vlm_client`, `test_ai_telemetry`, `test_alerting`, `test_migration_contract...`), dùng `respx` giả lập HTTP.
- **Katalon Studio (PA5):** kịch bản record&playback cho UC01, UC02 — báo cáo học thuật.

Verification vs Validation trong repo? [Cốt lõi]

Trả lời. **Verification** (đúng đặc tả): `tsc + vitest + pytest` tự động trong CI. **Validation** (đúng cái người dùng cần): Playwright mô phỏng hành trình thật + UAT beta mời người đọc manga thử.

Defect log trống (0 defect) --- có đáng ngờ không?

Trả lời. Thừa nhận: test report ghi 61/61 Pass, defect log rỗng. Nếu bị hỏi “test cho có?” → giải thích test bắt lỗi **trong lúc phát triển** (CI báo đỏ là fix ngay, không lọt vào report cuối); và có thể **trích 1–2 bug thật từ git history** (vd bug HTTP/2 broken pipe ở `database.py`, bug “trang ma” khi restart) làm ví dụ `found` → `fixed` → `retested`.

9. Triển khai & Vận hành

Bốn nền tảng triển khai? [Cốt lõi]

Trả lời. **Vercel** (frontend, CDN toàn cầu); **Render** (backend Docker, free 512MB, Singapore, auto-deploy main, healthcheck /health); **HuggingFace Spaces** (AI Module ~4GB, deploy thủ công); **Supabase** (Postgres + pgvector + Auth + Storage, AWS us-east-1). Ba dịch vụ đầu auto-deploy khi push main.

Cold start là gì? Nhóm xử lý ra sao? [Cốt lõi]

Trả lời. Render/HF Spaces free tier **ngủ sau ~15 phút nhàn rỗi**; request đầu tiên chậm ~30–60s (cold start). Giảm bằng **keep-alive** (GitHub Action ping định kỳ) + `api.ts` **retry khi cold-start** + backend degrade an toàn (health báo degraded thay vì sập).

Có backup/khôi phục không? [Cốt lõi]

Trả lời. Supabase **PITR (Point-in-Time Recovery) 7 ngày** (WAL liên tục); `pg_dump` thủ công trước mỗi migration rủi ro; Storage bật versioning; ảnh gốc soft-delete 30 ngày. Có runbook sự cố (rollback Render, restart pool, xoay key Gemini).

Scale nhiều replica thế nào (nếu tải tăng)? [Trung thực]

Trả lời. Hiện chạy 1 replica (free tier); cancel + event-bus + rate-limit lưu trong RAM. Đã có sẵn đường `REDIS_URL`: set biến này thì state chuyển sang Redis, cancel/event-bus/rate-limit hoạt động xuyên nhiều replica — cùng API, chỉ khác chỗ lưu. Đây là điểm thiết kế trưởng thành đáng khoe.

10. Bảo mật toàn repo (tổng hợp)

Kể các lớp bảo mật trong mã nguồn. [Cốt lõi]

Mối đe dọa	Phòng thủ (file)
XSS đánh cắp token	HTTP-only cookie (<code>auth.py</code>)
CSRF	Double-submit token + SameSite (<code>middleware.py</code>)
Rò rỉ chéo (IDOR)	<code>filter_user_id</code> RAG + RLS (<code>rag.py</code>)
SSRF (import ảnh)	Domain allowlist (<code>scraper.py</code>)
Giả mạo MIME	<code>Pillow verify()</code> (<code>image_validation.py</code>)
Spam/abuse	Rate limit SlowAPI (<code>rate_limit.py</code>)
Bot	Turnstile CAPTCHA (<code>captcha.py</code>)
Header attack	CSP/HSTS/nosniff/X-Frame (<code>middleware.py</code>)
Quá tải/OOM	Semaphore + BodySizeLimit
Double-charge	Idempotency key (<code>idempotency.py</code>)

Đây là **Defense in Depth** — nhiều lớp, từng lớp này còn lớp khác. Nguyên tắc LN12: enforce bằng code, không chỉ hứa.

11. Câu bẫy toàn repo & điểm yếu phải thủ

- Backend chạy đa luồng nhưng Render 1 process?** → semaphore giới hạn 10 luồng; có đường Redis để scale replica.
- CI không chạy khi chỉ sửa backend** (paths: `frontend/**`) → thừa nhận, hướng thêm `backend/**`.
- Embedding sinh ở backend gọi Gemini (tồn quota)** → đổi lấy đồng bộ 1 nhà cung cấp + không nạp thêm model vào AI Module; có key rotation + cache.

4. **YOLO thiếu artifact gốc** → chỉ có `best.pt`; hướng lưu `results.csv`.
5. **CER 6,1% chỉ đo in-distribution** → OOD chưa đo số; bù bằng Gemini self-correction + Studio QC.
6. **Defect log rỗng** → trích bug thật từ git history nếu bị hỏi.
7. **Migration chạy tay** (free tier không có pre-deploy hook) → mọi migration idempotent nên an toàn; nâng Starter \$7 để tự động.
8. **Tài liệu PA ghi ChromaDB/GPT-4** → tiến hoá RUP; sản phẩm cuối pgvector/Gemini tốt hơn; Risk R4 ghi rõ.
9. **Model monitoring đầy đủ (MLOps)?** → có nền `model_monitor` + `/metrics` Prometheus; drift dashboard là roadmap W&B/MLflow.
10. **Bản quyền** → train Manga109-s (license nghiên cứu); user content Private + kiểm duyệt + DMCA roadmap; Risk R5.

12. Phụ lục --- Cheat-sheet số liệu repo

Số liệu & sự thật repo phải thuộc

- **Cấu trúc:** 3 dịch vụ (frontend Vercel / backend Render 512MB / AI Module HF ~4GB) + Supabase.
- **Backend:** 24 router, 26 service; API prefix `/v1`; semaphore 10 luồng; embedding 768 chiều; RAG top-k=5, min-sim=0.55.
- **Frontend:** Next.js 16.2.6, React 19.2.4; ~24 route; 3 context; 3 theme; PWA.
- **CI:** 5 job (test/lint/build/e2e/backend); Playwright chromium+mobile; branch main/dev/duong.
- **Test:** pyramid — Vitest+RTL, Playwright 7 spec, pytest 14 file, Katalon (PA5 UC01/UC02).
- **Mô hình:** OCR CER 6,1% (12,7% → 6,1%); YOLO mAP@0.5 95,3%; test 956 ảnh/14.130 vùng chữ; Kaggle T4.
- **DB:** Supabase Postgres + pgvector + pg_trgm; ~18 migration v1–v9; 4 storage bucket; RLS.
- **Deploy:** auto-deploy main; healthcheck `/health`; PITR 7 ngày; keep-alive chống cold start.